

目录

- 1 作业要求**
- 2 实验分析**
 - 2.1 登录网站
 - 2.2 分析网页结构
 - 2.3 分析页面元素
- 3 爬虫设计**
 - 3.1 整体架构设计
 - 3.2 配置管理设计
 - 3.3 Cookie管理与半自动登录设计
 - 3.3.1 Cookie持久化设计
 - 3.3.2 基于Selenium的半自动登录设计
 - 3.4 代理池设计
 - 3.4.1 代理池核心结构
 - 3.4.2 代理选择策略
 - 3.4.3 代理健康管理
 - 3.4.4 代理池使用说明
 - 3.5 爬虫核心逻辑设计
 - 3.5.1 URL构建逻辑
 - 3.5.2 页面获取与重试机制
 - 3.5.3 数据解析逻辑
 - 3.5.4 异常处理与失败页面保存
 - 3.5.5 主爬取流程
 - 3.6 主程序流程控制
 - 3.7 数据存储设计
- 4 爬虫运行结果**
 - 4.1 运行方法
 - 4.2 执行结果
- 5 作业总结与心得体会**
- 6 附录A：提交材料结构说明**
- 7 附录B：爬取文件的完整结果**
- 8 附录C：爬虫主程序代码实现**

作业一 爬取北京链家官网二手房数据

1 作业要求

从爬取北京链家官网二手房数据，具体爬取信息及存储要求如下：

- 要求爬取东城、西城、海淀、朝阳四个城区的二手房数据（每个城区爬取前3页），提取城区、房源编号、小区名称、平米数、总价、单价（位置见图 1-1）；
- 将信息保存在同一个 csv 文件中；



图 1-1 待爬取内容位置

2 实验分析

2.1 登录网站

访问目标网站（<https://bj.lianjia.com/ershoufang/>），如图 2-1 所示：

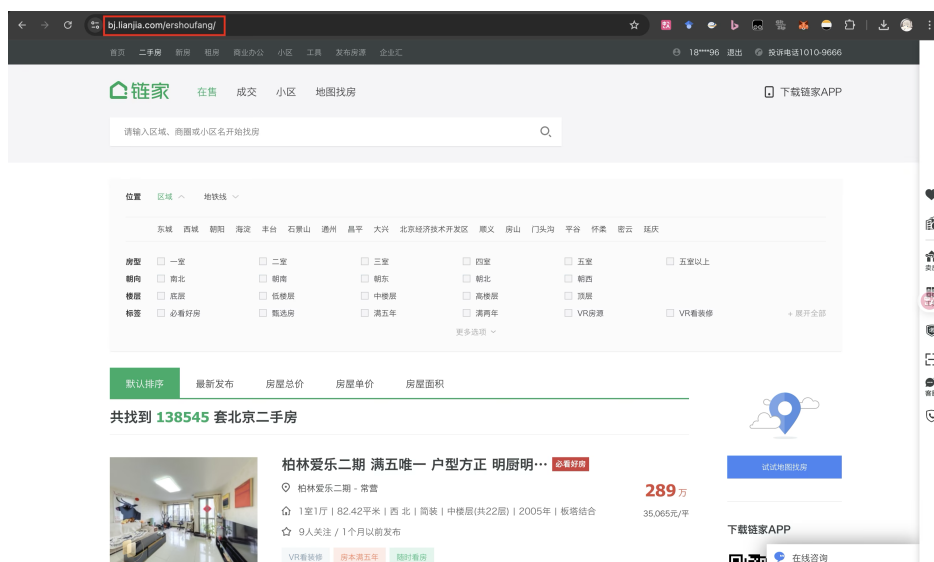


图 2-1 登录目标网站

链家网站需要手机号进行注册和登录，过程略。由于需要手机号绑定，爬取时采用账号池对我来说不可行。

2.2 分析网页结构

1. 省区域限制通过三级域名 `xxx.lianjia.com` 限制，例如本次作业中，北京链家官网二手房对应 `https://bj.lianjia.com/`；
2. 区域限制通过后缀 url 控制，如图 2-2 所示，在我点击选择“海淀”后，上方 url 变为 `https://bj.lianjia.com/ershoufang/haidian/`，同样可以验证其他区域与 url 的映射关系为“区域名”-“拼音”，除了极少数可能存在重复的情况。对于作业中涉及的几个城区，映射关系如下：
 - 东城 - `dongcheng`；
 - 西城 - `xicheng`；
 - 海淀 - `haidian`；
 - 朝阳 - `chaoyang`；
 - 其中，baseurl 均为 `https://bj.lianjia.com/ershoufang/`；

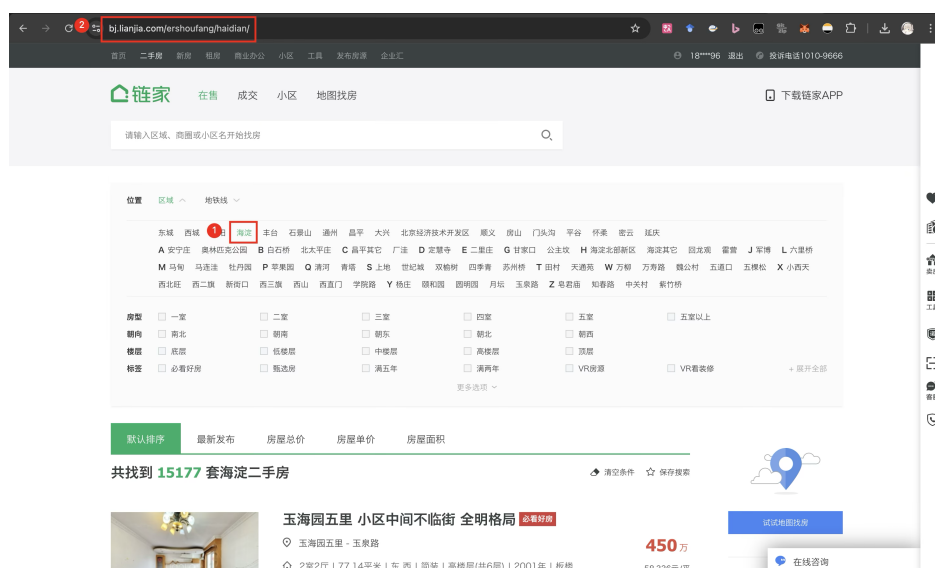


图 2-2 确定区域与 url 的映射关系

3. 接下来确定分页逻辑，浏览网页后发现，一页的内容限制为 30 条，点击下一页，可以发现上方的 url 后新增 `pg2`，如图 2-3 所示，可以验证，页面与 url 的映射关系如下：
 - 区域 x 的第 y 页数据 - `x/pgy`；（除第一页外）
 - 其中，baseurl 均为 `https://bj.lianjia.com/ershoufang/`；

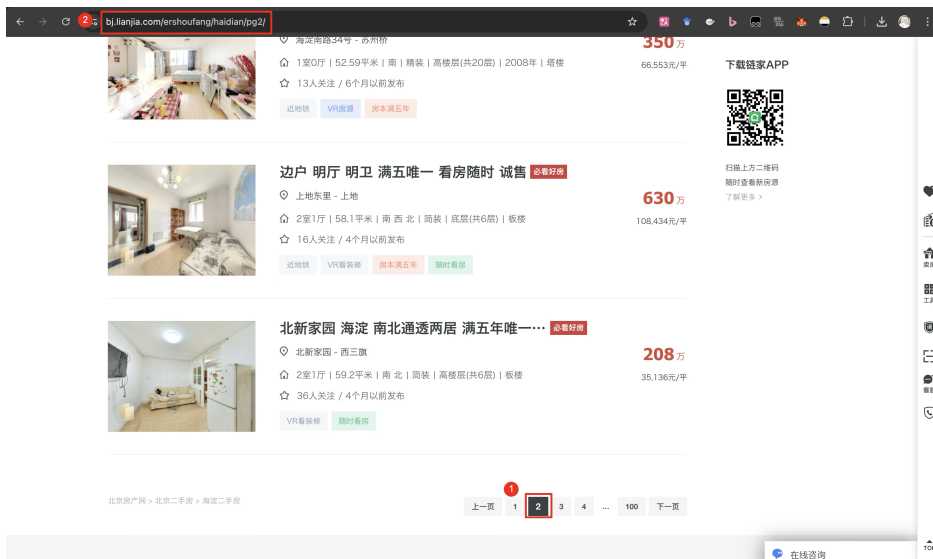


图 2-3 确定分页逻辑

4. 确定有效页面数：（本次作业中可以先不考虑）
 - (a) 首先，链家默认支持 1 ~ 100 页的页数范围；
 - (b) 首页中显示“共找到 15175 套海淀二手房”，那么根据一页 30 条，有效页面有 $15175 / 30$ 向上取整为 506，但实际上访问到 34 页时就已经没有有效数据了，如图 2-4 所示；
 - (c) 因此，实际实现中，可以通过检查页面元素来确定当前页是否为有效页面，如果不是则停止；

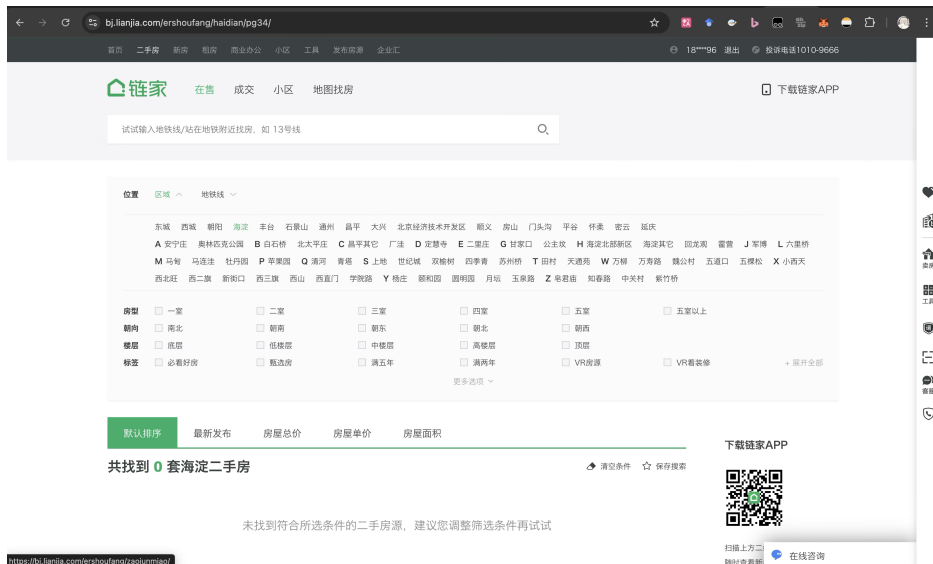


图 2-4 确定有效页面

2.3 分析页面元素

1. 通过 `fn` + `F12` 打开 DevTools，切换到 Network 面板，刷新页面：
2. 查看请求：点击主文档请求，观察 Response，如图 2-5 所示，可以看到我们需要的元素均在主文档请求中；

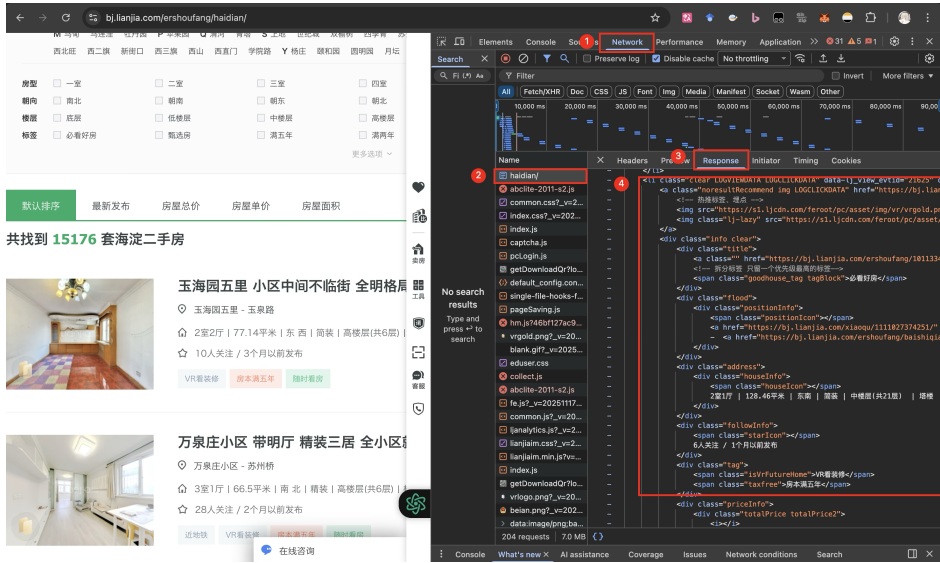


图 2-5 查看主页面请求

3. 定位数据元素：
 - (a) 房源列表容器：对应 `ul.sellListContent`，如图 2-6 所示；
 - (b) 房源编号：在房源链接URL中 (如 `/ershoufang/101123456789.html`)，如图 2-7 所示；
 - (c) 小区名称：在 `.positionInfo a` 中，如图 2-8 所示；
 - (d) 平米数：在 `.houseInfo` 中，需要解析，如图 2-9 所示；
 - (e) 总价：在 `.totalPrice span` 中，如图 2-10 所示；
 - (f) 单价：在 `.unitPrice span` 中，如图 2-11 所示；



图 2-6 房源列表容器

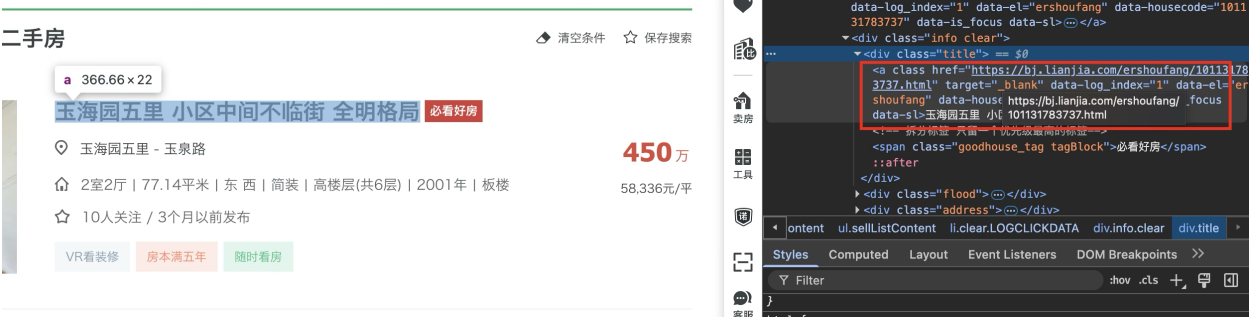


图 2-7 房源编号



图 2-8 小区名称



图 2-9 平米数



图 2-10 总价



图 2-11 单价

4. 反爬机制观察：

- (a) 如果高频切换页面，会触发如图 2-12 所示的人机验证，由于验证方式为图案点击，传统的过码工具无效，所以在爬取时应尽可能减少触发人机验证；
- (b) 此外，由于后续需要爬虫通过发送请求访问目标网站，为将该请求伪造为一个从浏览器发出的请求，还需要观察 需要 User-Agent, Referer，以及 Cookie 等。打开 Network，随便找一个请求，观察其请求头内容，如图 2-13 所示，其中 Cookie 信息由于安全需要进行打码处理；
 - i. `user-agent`：Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36；这里不需要完全一致，可以伪造多个轮询；
 - ii. `referer`：<https://bj.lianjia.com/ershoufang/haidian/>；
 - iii. 敏感认证凭证包括 `login_ucid`，`lianjia_token`，`lianjia_token_secure` 和 `security_ticket`；



图 2-12 触发人机验证

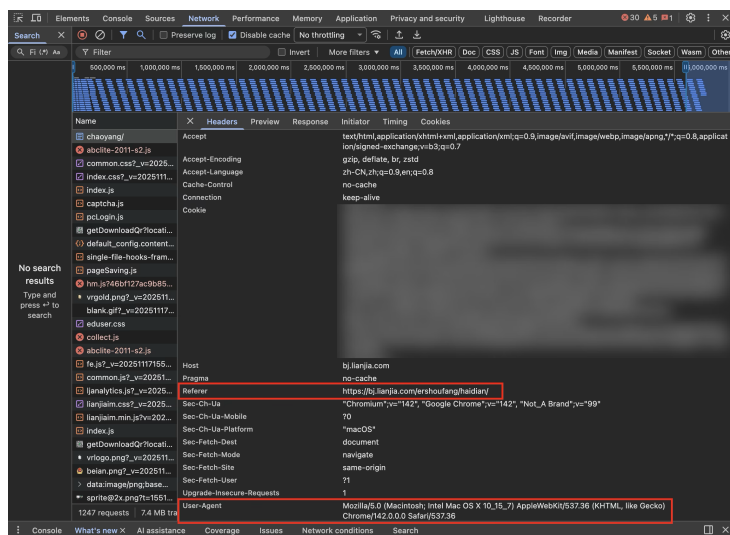


图 2-13 请求头内容

3 爬虫设计

3.1 整体架构设计

爬虫系统采用模块化设计，将不同功能拆分到独立模块中，提高代码可维护性和复用性。整体架构包括以下核心模块：

1. **配置模块 (config.py)**：集中管理爬虫参数配置
2. **Cookie管理模块 (cookie_manager.py)**：负责Cookie的持久化存储和加载
3. **自动登录模块 (auto_login.py)**：基于Selenium实现的自动化登录
4. **代理池模块 (proxy_pool.py)**：提供IP代理轮换功能，应对反爬策略
5. **爬虫核心模块 (crawler.py)**：实现页面抓取、数据解析和存储
6. **主程序模块 (main.py)**：协调各模块，控制整体爬取流程

3.2 配置管理设计

考虑到爬虫程序的可扩展性和可维护性，我将所有有可能需要调整的参数集中到独立的配置文件 `config.py` 文件中管理。主要配置项包括：

```
1  # 基础URL
2  BASE_URL = "https://bj.lianjia.com/ershoufang/"
3
4  # 城区映射：城区名 -> URL路径
5  DISTRICTS = {
6      "东城": "dongcheng",
7      "西城": "xicheng",
8      "海淀": "haidian",
9      "朝阳": "chaoyang",
10 }
11
12 # 每个城区爬取页数
13 PAGES_PER_DISTRICT = 3
14
15 # 请求头配置
16 HEADERS = {
17     "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) ...",
18     "Referer": "https://bj.lianjia.com/ershoufang/",
19     "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,...",
20 }
21
22 # 反爬策略配置
23 REQUEST_DELAY = 2.0          # 请求间隔 (秒)
24 MAX_CONSECUTIVE_FAILURES = 3 # 连续失败阈值
25 PAGE_RETRY_COUNT = 3         # 单页重试次数
26 RETRY_DELAY = 3.0            # 重试间隔 (秒)
```

对于城区映射关系，采用字典结构存储城区名称到URL路径的映射。使得当需要扩展到更多城区时，只需在字典中添加新的键值对即可，无需修改爬虫的核心逻辑代码。

在反爬策略配置方面，通过 `REQUEST_DELAY` 参数控制请求间隔时间。经过前期分析，链家网站在检测到高频访问时会触发人机验证，因此设置了2秒的请求间隔，在保证爬取效率的同时，有效降低触发反爬机制的风险。

此外，为了应对网络波动或临时性的服务器错误，设计了三层容错机制

1. 首先是单个页面的重试机制（`PAGE_RETRY_COUNT`），当单次请求失败时会自动重试最多3次；
2. 其次是连续失败检测机制（`MAX_CONSECUTIVE_FAILURES`），当连续3个页面都获取失败时，程序会判断可能遇到了严重问题（如Cookie失效或IP被封禁），此时会停止爬取并保存已获取的数据；
3. 最后是重试延迟机制（`RETRY_DELAY`），每次重试前等待3秒，给服务器和网络环境一定的恢复时间。这种多层次的容错设计大大提高了爬虫在复杂网络环境下的稳定性和健壮性。

3.3 Cookie管理与半自动登录设计

3.3.1 Cookie持久化设计

由于链家网站需要登录才能访问完整数据，Cookie管理模块实现了Cookie的保存和加载功能，实现登录状态的持久化存储。

```
1 def load_cookies_from_file(filepath: str = "cookies.json") -> dict:
2     """从JSON文件加载cookies"""
3     if not os.path.exists(filepath):
4         return {}
5
6     with open(filepath, 'r', encoding='utf-8') as f:
7         cookies = json.load(f)
8     return cookies
9
10 def save_cookies_to_file(cookies: dict, filepath: str = "cookies.json"):
11     """保存cookies到JSON文件"""
12     with open(filepath, 'w', encoding='utf-8') as f:
13         json.dump(cookies, f, indent=2, ensure_ascii=False)
```

如图 3-1 所示，Cookie 文件包含了所有从浏览器中提取的 Cookie 键值对。如图 3-2 所示，程序启动时会首先尝试从文件中加载 Cookie，只有在文件不存在或 Cookie 失效时才会触发登录流程。

[illegible]

图 3-1 Cookie 的持久化存储

```
=====
链家二手房爬虫
=====
Cookie文件不存在: cookies.json
▲ 未检测到Cookie文件
启动自动登录流程...
```

图 3-2 启动爬虫前检查 Cookie

3.3.2 基于Selenium的半自动登录设计

为了获取有效的登录 Cookie，我通过 Python 的 Selenium 库，实现了半自动登录模块。该模块自动打开浏览器并导航到登录页面，然后等待用户手动完成登录操作（包括人机验证），最后自动提取 Cookie 并保存。

这里目前未实现的自动化登录的理由如下:

1. 链家网由于业务逻辑较复杂，很难通过抓包找到并逆向出对应的登录 API；
2. 即使是通过手机号 + 密码登录，链家网也要求通过手机号接收验证码，无法免去用户的手动操作；
3. 接收验证码前有时还会出发人机验证，难以自动过码；

因此，由程序自动完成可自动化的部分（打开浏览器、导航到登录页），而将难以自动化的部分（手机验证码、人机验证）交由用户手动完成。

核心实现逻辑:

在反检测机制方面，现代网站通常会通过检测 `navigator.webdriver` 属性、自动化扩展等特征来识别 Selenium 控制的浏览器。为了降低被检测的风险，我在初始化 WebDriver 时配置了多项反检测参数，包括：

- 通过 `--disable-blink-features=AutomationControlled` 参数禁用了Blink引擎的自动化控制特征；
- 通过 `excludeSwitches` 选项移除了 `enable-automation` 开关，避免浏览器显示"正在受到自动化软件控制"的提示；
- 使用CDP（Chrome DevTools Protocol）命令在页面加载前注入JavaScript代码，将 `navigator.webdriver` 属性重新定义为 `undefined`，从而使网站无法通过该属性判断浏览器是否被自动化工具控制

实际测试后发现通过综合使用这些措施，能有效降低被识别为爬虫的风险。

```

1 class AutoLogin:
2     def __init__(self, headless=False):
3         """初始化WebDriver并配置反检测参数"""
4         chrome_options = Options()
5         # 禁用自动化控制特征
6         chrome_options.add_argument('--disable-blink-features=AutomationControlled')
7         chrome_options.add_experimental_option('excludeSwitches', ['enable-automation'])
8
9         self.driver = webdriver.Chrome(options=chrome_options)
10        # 隐藏webdriver属性
11        self.driver.execute_cdp_cmd('Page.addScriptToEvaluateOnNewDocument',
12        {
13            'source': 'Object.defineProperty(navigator, "webdriver", {get: ()
=> undefined})'
14        })

```

如图 3-3 所示，登录流程设计采用轮询检测机制，通过检查关键Cookie的存在来判断登录是否成功。同时，为了避免程序无限等待，设计了超时保护机制。登录等待的默认超时时间设置为120秒，若超时仍未检测到登录成功，程序会返回失败状态并提示用户重试，具体代码实现如下：

```

1 def login_manual(self, wait_time=120):
2     """等待用户手动完成登录"""
3     self.driver.get("https://bj.lianjia.com/")
4
5     # 自动点击登录按钮
6     login_btn = WebDriverWait(self.driver, 10).until(
7         EC.element_to_be_clickable((By.LINK_TEXT, "登录"))
8     )
9     login_btn.click()
10
11    # 轮询检测登录状态
12    start_time = time.time()
13    while time.time() - start_time < wait_time:
14        current_cookies = self.driver.get_cookies()

```



```

15         cookie_dict = {c['name']: c['value'] for c in current_cookies}
16
17         # 检测关键认证Cookie
18         has_token = 'lianjia_token' in cookie_dict and
len(cookie_dict['lianjia_token']) > 10
19         has_ucid = 'login_ucid' in cookie_dict and
len(cookie_dict['login_ucid']) > 10
20
21         if has_token or has_ucid:
22             # 登录成功, 保存Cookie
23             cookies = self.get_cookies()
24             self.save_cookies(cookies)
25             return cookies
26
27         time.sleep(1)
28
29     return None # 超时失败

```

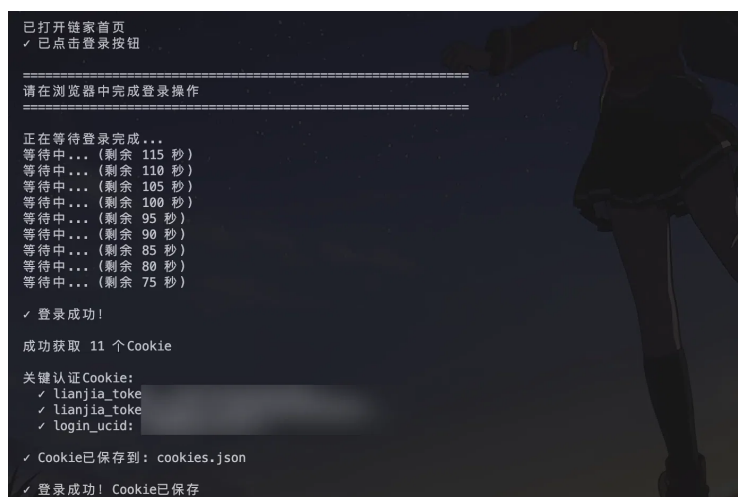


图 3-3 半自动登录流程

3.4 代理池设计

为了应对可能的 IP 封禁, 我设计了灵活的代理池管理系统, 支持多种代理选择策略和自动故障检测,

在代理配置管理方面, 使用 JSON 配置文件中如图 3-4 所示。包含代理的基本信息 (主机、端口、协议) 以及认证信息 (用户名、密码) 和启用状态。程序启动时会自动加载配置文件, 并为每个代理初始化状态跟踪对象, 记录其成功次数、失败次数、最后使用时间等信息。

```

1  {
2    "proxies": [
3      {
4        "name": "Proxy1",
5        "host": "127.0.0.1",
6        "port": 8080,
7        "username": "user",
8        "password": "pass",
9        "type": "http",
10       "enabled": true
11     },
12     {
13       "name": "Proxy2",
14       "host": "127.0.0.1",
15       "port": 8080,
16       "username": "user",
17       "password": "pass",
18       "type": "http",
19       "enabled": true
20     }
21   ],
22   "config": {
23     "strategy": "random",
24     "retry_times": 3,
25     "timeout": 10,
26     "test_url": "https://bj.lianjia.com/",
27     "min_interval": 1.0
28   }
29 }

```

图 3-4 代理池配置文件

3.4.1 代理池核心结构

```

1  class ProxyPool:
2      def __init__(self, proxy_file: str = "proxy_pool.json"):
3          """从配置文件加载代理列表"""
4          self.proxies = []          # 代理配置列表
5          self.proxy_status = {}     # 代理状态统计
6          self.load_proxies()

```

3.4.2 代理选择策略

支持四种代理选择策略，适应不同的使用场景：

- 轮询策略（round_robin）适合负载均衡，确保每个代理使用次数相近；
- 随机策略（random）可以增加访问模式的不确定性，降低被检测的风险；
- 最少使用策略（least_used）适合在代理性能差异较大时使用，优先使用负载较轻的代理；
- 最佳性能策略（best_performance）则根据历史成功率选择表现最好的代理；

对应的策略代码实现如下：

```

1  def get_proxy(self, strategy: str = "round_robin") -> Optional[Dict]:
2      """
3      根据策略获取代理
4      - round_robin: 轮询（默认）
5      - random: 随机选择
6      - least_used: 选择使用次数最少的
7      - best_performance: 选择成功率最高的

```

```

8         """
9         available_proxies = [
10             p for p in self.proxies
11             if p.get('enabled', True) and
12                self.proxy_status.get(p['name'], {}).get('available', True)
13         ]
14
15         if strategy == "random":
16             proxy_config = random.choice(available_proxies)
17         elif strategy == "best_performance":
18             proxy_config = max(available_proxies,
19                               key=lambda p: self._get_success_rate(p['name']))
20         # ... 其他策略
21
22         return self._build_proxy_dict(proxy_config)

```

3.4.3 代理健康管理

通过记录每个代理的成功/失败次数，实现了自动故障检测和转移机制。每次使用代理后，程序都会记录请求是成功还是失败。如果某个代理连续失败3次，系统会自动将其标记为不可用，后续的代理选择过程会跳过该代理。

```

1 def mark_failure(self, proxy_dict: Dict):
2     """标记代理失败，连续失败3次则自动禁用"""
3     proxy_id = proxy_dict['_config']['name']
4     self.proxy_status[proxy_id]['fail'] += 1
5
6     if self.proxy_status[proxy_id]['fail'] >= 3:
7         self.proxy_status[proxy_id]['available'] = False
8         print(f"⚠️ 代理 {proxy_id} 连续失败，暂时禁用")

```

3.4.4 代理池使用说明

我在实验中采用的代理池是通过在云服务器上配置 squid 接管 http 流量，本机使用云服务器的 IP 代理访问。不过实际测试中发现：

- 不同 IP 通过同一账号访问网页，容易触发 Cookie 刷新机制，或触发风控，如图 3-5 所示；
- 云服务器厂商提供的云服务器的 IP 因为某些特殊性更容易被网站检测；

```
4 URL: https://bj.lianjia.com/ershoufang/dongcheng/pg3/
5 时间: 2025-11-21 22:39:30
6 状态码: 200
7 失败原因: 未解析到房源数据 (可能是Cookie失效)
8 =====
9 -->
10 <!DOCTYPE html>
11 <html lang="en">
12 <head>
13   <meta charset="UTF-8"/>
14   <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
15   <meta name="description" content="Captcha"/>
16   <link rel="icon" href="//s1.ljcdn.com/hip-static/favicon.ico" type="image/x-icon"/>
17   <link rel="stylesheet" href="//s1.ljcdn.com/hip-static/css/main.css">
18   <title>CAPTCHA</title>
19   <style>
20     #captcha {
21       width: auto;
22       display: flex;
23       flex-direction: column;
24       justify-content: center;
25       align-items: center;
26       margin-top: 10px;
27     }
28   </style>
29 </head>
30 <body>
31 <main id="main">
32   <div id="container">
33     
34     <div class="title">人机验证</div>
35
36     <div id="captcha">
37       <div id="loading-tip">加载中, 请稍后...</div>
38     </div>
39   </div>
40   <footer id="site-footer">
41     <div class="footer-content">
42       
```

图 3-5 触发风控的情况

上图是程序在运行时提取失败时自动保存的 HTML 页面，状态码为 200 虽然正常响应，但从网页内容来看，没有有效的房源容器，并且明显触发了人机验证。

因此，最好的方法是使用一些市面上稳定的代理池，并且配合帐号池使用。

3.5 爬虫核心逻辑设计

3.5.1 URL构建逻辑

根据城区代码和页码动态构建目标URL：

```
1 def build_url(district_code: str, page: int) -> str:
2     """
3     构建目标URL
4     第1页: https://bj.lianjia.com/ershoufang/haidian/
5     第N页: https://bj.lianjia.com/ershoufang/haidian/pgN/
6     """
7     if page == 1:
8         return f"{BASE_URL}{district_code}/"
9     return f"{BASE_URL}{district_code}/pg{page}/"
```

3.5.2 页面获取与重试机制

为提高爬虫稳定性，底层的 `fetch_page` 函数负责执行单次HTTP请求，而上层的 `fetch_page_with_retry` 函数则封装了重试逻辑，实现了重试机制，对应代码如下：

```
1 def fetch_page_with_retry(url: str, cookies: dict = None,
2                             proxy_pool: ProxyPool = None) -> tuple:
3     """带重试机制的页面获取"""
4     for attempt in range(PAGE_RETRY_COUNT + 1):
5         html, status_code, error = fetch_page(url, cookies, proxy_pool)
6
7         # 成功则立即返回
8         if status_code == 200 and html:
9             return html, status_code, None, attempt
10
11        # 失败则等待后重试
12        if attempt < PAGE_RETRY_COUNT:
13            print(f"    ♡ 第 {attempt + 1} 次重试 (等待 {RETRY_DELAY} 秒) ...")
14            time.sleep(RETRY_DELAY)
15
16    return last_html, last_status, last_error, PAGE_RETRY_COUNT
```

`fetch_page_with_retry` 函数实现了智能重试逻辑。它会尝试最多4次请求（1次初始请求 + 3次重试），每次失败后会输出重试提示并等待一定时间（由 `RETRY_DELAY` 配置）再进行下次尝试。这个等待时间的设置很重要：如果等待时间太短，可能服务器或网络问题还未恢复；如果等待时间太长，则会显著降低爬取效率。经过权衡，设置为3秒是一个较为合理的选择。

实际的页面获取支持代理和直连两种模式：当代理池参数传入时，程序会调用 `fetch_with_proxy` 函数，该函数会从代理池中获取一个可用代理，并使用该代理发送请求；当代理池参数为空时，程序直接使用原始IP发送请求。具体代码如下：

```
1 def fetch_page(url: str, cookies: dict = None,
2                 proxy_pool: ProxyPool = None) -> tuple:
3     """获取页面HTML (支持代理池) """
4     try:
5         if proxy_pool:
6             # 使用代理池
7             response = fetch_with_proxy(url, proxy_pool,
8                                         headers=HEADERS,
9                                         cookies=cookies,
10                                        timeout=10)
11        else:
12            # 直连
13            response = requests.get(url, headers=HEADERS,
14                                    cookies=cookies, timeout=10)
15
16    return response.text, response.status_code, None
```

```
17     except requests.RequestException as e:
18         return "", None, str(e)
```

3.5.3 数据解析逻辑

使用 BeautifulSoup 库解析 HTML，提取房源数据，对于所需字段的提取逻辑如下（字段查找逻辑我已经在 [分析页面元素](#) 一节详细说明）：

- 对于直接存在于HTML标签文本中的数据（如小区名称、总价），使用 `.text.strip()` 方法即可获取并去除首尾空白；
- 对于嵌入在复杂文本中的数据（如平米数、房源编号、单价），则需要使用正则表达式进行提取。
 - 例如，房源编号从链接URL `/ershoufang/101123456789.html` 中提取，使用正则表达式 `r'/ershoufang/(\d+)\.html'` 可以精确匹配数字部分；
- 平米数从 "2室1厅 | 89.3平米 | 南 | 精装" 这样的复合文本中提取，通过正则表达式 `r'([\d.]+) 平米'` 可以匹配小数格式的平米数。

对应的代码实现如下：

```
1 def parse_page(html: str, district_name: str) -> list:
2     """解析页面，提取房源数据"""
3     houses = []
4     soup = BeautifulSoup(html, 'html.parser')
5
6     # 1. 定位房源列表容器
7     list_content = soup.select_one('ul.sellListContent')
8     if not list_content:
9         return houses
10
11    # 2. 遍历每个房源项
12    for item in list_content.select('li.clear'):
13        house = {'城区': district_name}
14
15        # 3. 提取房源编号（从链接URL中解析）
16        title_link = item.select_one('.title a')
17        if title_link and title_link.get('href'):
18            house['房源编号'] = parse_house_id(title_link['href'])
19
20        # 4. 提取小区名称
21        position_info = item.select_one('.positionInfo a')
22        house['小区名称'] = position_info.text.strip() if position_info else ""
23
24        # 5. 提取平米数（需要正则解析）
25        house_info = item.select_one('.houseInfo')
```

```

26         house['平米数 (单位:平米)'] = parse_area(house_info.text) if house_info
else ""
27
28         # 6. 提取总价和单价
29         total_price = item.select_one('.totalPrice span')
30         house['总价(单位:万)'] = total_price.text.strip() if total_price else
""
31
32         unit_price = item.select_one('.unitPrice span')
33         house['单价(单位:元/平)'] = parse_unit_price(unit_price.text) if
unit_price else ""
34
35         if house['房源编号']:
36             houses.append(house)
37
38     return houses

```

辅助解析函数使用正则表达式提取数值：

```

1  def parse_house_id(href: str) -> str:
2      """从URL提取房源编号: /ershoufang/101123456789.html -> 101123456789"""
3      match = re.search(r'/ershoufang/(\d+)\.html', href)
4      return match.group(1) if match else ""
5
6  def parse_area(house_info: str) -> str:
7      """从房屋信息提取平米数: "2室1厅 | 89.3平米 | ..." -> "89.3" """
8      match = re.search(r'([\d.]+)平米', house_info)
9      return match.group(1) if match else ""
10
11 def parse_unit_price(price_text: str) -> str:
12     """解析单价: "单价65,123元/平米" -> "65123" """
13     clean_text = price_text.replace(',', '') # 移除千分位逗号
14     match = re.search(r'(\d+)', clean_text)
15     return match.group(1) if match else ""

```

在容错处理方面，每个字段的提取都包含了完整的异常处理逻辑。使用条件表达式（如 `position_info.text.strip() if position_info else ""`）确保即使某个元素不存在，程序也不会崩溃，而是记录空字符串。同时，在房源项遍历的最外层使用try-except捕获所有可能的异常，确保单个房源的解析失败不会影响其他房源的处理。

最后，只有当房源编号存在时才将该房源添加到结果列表，这个设计基于房源编号是唯一标识的考虑，避免将无效数据写入输出文件，使得爬虫即使在遇到页面结构异常时也能尽可能多地提取有效数据。

3.5.4 异常处理与失败页面保存

爬虫运行时往往会遇到 Cookie失效、网络问题、IP被封禁、触发人机验证等多种问题。为便于调试和分析失败原因，将获取失败的页面保存到本地：

```
1 def save_failed_page(html: str, url: str, reason: str, status_code: int =  
None):  
2     """保存失败页面到本地，便于分析"""  
3     os.makedirs(FAILED_PAGES_DIR, exist_ok=True)  
4  
5     timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")  
6     url_part = url.replace(BASE_URL, "").replace("/", "_").strip("_") or  
"index"  
7     filename = f"{timestamp}_{url_part}.html"  
8     filepath = os.path.join(FAILED_PAGES_DIR, filename)  
9  
10    # 添加调试信息头  
11    debug_info = f"""<!--  
12    失败页面调试信息  
13    =====  
14    URL: {url}  
15    时间: {datetime.now().strftime("%Y-%m-%d %H:%M:%S")}  
16    状态码: {status_code or "N/A"}  
17    失败原因: {reason}  
18    =====  
19    -->  
20    """  
21  
22    with open(filepath, 'w', encoding='utf-8') as f:  
23        f.write(debug_info + (html or "<!-- 页面内容为空 -->"))
```

3.5.5 主爬取流程

爬虫的主控制逻辑集成了所有功能模块，是整个爬虫系统的控制中枢：

1. 爬取顺序的设计上，我采用了双层循环的嵌套结构：外层循环遍历城区字典，内层循环遍历每个城区的页码；
2. 失败检测机制设计上，程序维护了一个 `consecutive_failures` 计数器，记录连续失败的页面数。当计数器达到阈值（配置为3）时，程序会判断可能遇到了系统性问题，会主动终止爬取流程。
3. Cookie失效检测设计上，我通过两种方式检测 Cookie 是否失效：
 - (a) HTTP状态码检测，当返回302（重定向到登录页）、403（禁止访问）或401（未授权）时，很可能是Cookie已过期或失效；
 - (b) 首页数据检测，如果第一页就无法解析出任何房源数据，通常也表明Cookie有问题。

检测到Cookie失效后，程序会设置 `need_relogin` 标志，并在返回时通知主程序，由主程序决定是否重新登录。

4. 数据保存策略上，程序在两个时机会保存数据：

- (a) 正常完成所有爬取后；
- (b) 因连续失败而提前终止时，确保即使爬取过程被中断，已经获取的数据也不会丢失。

对应的代码实现如下：

```
1 def crawl(cookies: dict = None, use_proxy: bool = False) -> tuple:
2     """执行爬虫主逻辑"""
3     all_houses = []
4     consecutive_failures = 0
5     need_relogin = False
6
7     # 初始化代理池（如果启用）
8     proxy_pool = ProxyPool() if use_proxy else None
9
10    # 双层循环：遍历城区和页码
11    for district_name, district_code in DISTRICTS.items():
12        for page in range(1, PAGES_PER_DISTRICT + 1):
13            url = build_url(district_code, page)
14
15            # 获取页面（带重试）
16            html, status_code, error, retries = fetch_page_with_retry(
17                url, cookies, proxy_pool
18            )
19
20            # 检查HTTP状态码
21            if status_code and status_code != 200:
22                save_failed_page(html, url, f"HTTP {status_code}",
status_code)
23                consecutive_failures += 1
24
25            # 检测Cookie失效
26            if status_code in [302, 403, 401]:
27                need_relogin = True
28
29            # 检查是否获取到内容
30            elif not html or error:
31                save_failed_page(html, url, error or "空页面")
32                consecutive_failures += 1
33
34            else:
35                # 解析页面
36                houses = parse_page(html, district_name)
37
38                if houses:
39                    all_houses.extend(houses)
40                    consecutive_failures = 0 # 重置失败计数
```

```

41         else:
42             # 首页无数据, 可能Cookie失效
43             if page == 1:
44                 need_relogin = True
45
46             # 检查是否需要停止
47             if consecutive_failures >= MAX_CONSECUTIVE_FAILURES:
48                 if all_houses:
49                     save_to_csv(all_houses, OUTPUT_FILE)
50                 return all_houses, need_relogin
51
52             # 请求间隔
53             time.sleep(REQUEST_DELAY)
54
55         # 保存结果
56         if all_houses:
57             save_to_csv(all_houses, OUTPUT_FILE)
58
59         return all_houses, need_relogin

```

3.6 主程序流程控制

主程序 `main.py` 协调各模块, 实现完整的爬取流程:

1. 程序启动后首先调用 `check_and_get_cookies` 函数, 该函数会尝试从本地文件加载Cookie, 如果Cookie文件不存在, 会启动登录流程;
2. 在开始爬取前, 程序会询问用户是否使用代理池;
3. 当爬虫执行完成后, 程序会检查 `need_relogin` 标志。如果该标志为真 (表明爬取过程中检测到Cookie失效), 程序询问用户是否重新登录。因此, 即使在爬取过程中Cookie过期, 用户也可以在不重启程序的情况下恢复爬取。

对应的代码实现如下:

```

1  def main():
2      # 1. Cookie检查与获取
3      cookies = check_and_get_cookies()
4      if not cookies:
5          print("\n无法获取Cookie, 程序退出")
6          return
7
8      # 2. 询问是否使用代理
9      use_proxy = input("\n是否使用代理池? (y/n): ").strip().lower() == 'y'
10
11     # 3. 执行爬虫
12     houses, need_relogin = crawl(cookies, use_proxy=use_proxy)
13
14     # 4. Cookie失效处理
15     if need_relogin:

```

```
16         relogin = input("\n检测到Cookie失效, 是否重新登录? (y/n):  
17     ").strip().lower()  
18         if relogin == 'y':  
19             # 删除旧Cookie, 重新登录  
20             os.remove("cookies.json")  
21             new_cookies = get_cookies_via_login()  
22  
23             # 使用新Cookie继续爬取  
24             if new_cookies:  
                crawl(new_cookies, use_proxy=use_proxy)
```

3.7 数据存储设计

按照作业要求, 我将爬取的数据保存为 CSV 格式, 方便后续分析:

```
1 def save_to_csv(data: list, filepath: str):  
2     """保存数据到CSV文件 (UTF-8 with BOM) """  
3     os.makedirs(os.path.dirname(filepath), exist_ok=True)  
4  
5     with open(filepath, 'w', newline='', encoding='utf-8-sig') as f:  
6         writer = csv.DictWriter(f, fieldnames=CSV_COLUMNS)  
7         writer.writeheader()  
8         writer.writerows(data)
```

4 爬虫运行结果

4.1 运行方法

我采用 uv 进行 Python 包管理，因此在程序根目录下直接运行：

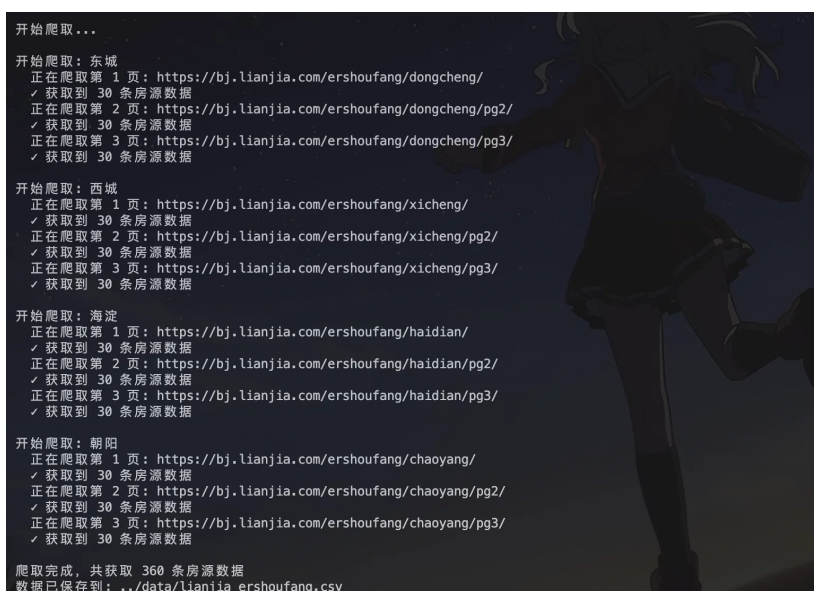
```
1 uv sync
```

即可自动配置所需 Python 环境和依赖。

然后执行：

```
1 cd src
2 uv run python main.py
```

即可启动爬虫程序，运行效果如图 4-1 所示。



```
开始爬取...
开始爬取：东城
正在爬取第 1 页：https://bj.lianjia.com/ershoufang/dongcheng/
✓ 获取到 30 条房源数据
正在爬取第 2 页：https://bj.lianjia.com/ershoufang/dongcheng/pg2/
✓ 获取到 30 条房源数据
正在爬取第 3 页：https://bj.lianjia.com/ershoufang/dongcheng/pg3/
✓ 获取到 30 条房源数据

开始爬取：西城
正在爬取第 1 页：https://bj.lianjia.com/ershoufang/xicheng/
✓ 获取到 30 条房源数据
正在爬取第 2 页：https://bj.lianjia.com/ershoufang/xicheng/pg2/
✓ 获取到 30 条房源数据
正在爬取第 3 页：https://bj.lianjia.com/ershoufang/xicheng/pg3/
✓ 获取到 30 条房源数据

开始爬取：海淀
正在爬取第 1 页：https://bj.lianjia.com/ershoufang/haidian/
✓ 获取到 30 条房源数据
正在爬取第 2 页：https://bj.lianjia.com/ershoufang/haidian/pg2/
✓ 获取到 30 条房源数据
正在爬取第 3 页：https://bj.lianjia.com/ershoufang/haidian/pg3/
✓ 获取到 30 条房源数据

开始爬取：朝阳
正在爬取第 1 页：https://bj.lianjia.com/ershoufang/chaoyang/
✓ 获取到 30 条房源数据
正在爬取第 2 页：https://bj.lianjia.com/ershoufang/chaoyang/pg2/
✓ 获取到 30 条房源数据
正在爬取第 3 页：https://bj.lianjia.com/ershoufang/chaoyang/pg3/
✓ 获取到 30 条房源数据

爬取完成，共获取 360 条房源数据
数据已保存到：../data/lianjia_ershoufang.csv
```

图 4-1 程序运行过程

4.2 执行结果

成功爬取到的数据默认保存在 `../data/lianjia_ershoufang.csv` 下，如图 4-2 所示：



1	城区, 房源编号, 小区名称, 平米数 (单位: 平米), 总价 (单位: 万), 单价 (单位: 元/平)
2	东城, 101132675987, 国瑞城东区, 69.01, 578, 83756
3	东城, 101133828145, 新中西里, 72.38, 670, 92568
4	东城, 101129411869, 广渠家园, 79.55, 650, 81710
5	东城, 101132480005, 地坛北里, 53.53, 536, 100131
6	东城, 101131524357, 广渠家园, 60.79, 465, 76493
7	东城, 101132603835, 蔡慧小区, 44.5, 300, 67416
8	东城, 101134144805, 地坛北门, 52.61, 499, 94849
9	东城, 101133592145, 民旺园, 55.27, 535, 96798
10	东城, 101132150391, 和平里五区, 55.12, 579, 105044
11	东城, 101134059409, 化工大院, 36.55, 445, 121752
12	东城, 101132979778, 西革新里112号院, 88.69, 699, 78814
13	东城, 101128711497, 望坛新苑, 65.62, 496, 75587
14	东城, 101134142498, 白桥大街, 57.86, 348, 60146
15	东城, 101134214716, 左安门内大街, 58.73, 340, 57893
16	东城, 101132225367, 冠城名敦道A区, 85.03, 520, 61155
17	东城, 101134172488, 景泰西里西区, 77.46, 450, 58095
18	东城, 101133576612, 鼎盛园, 81.91, 480, 58601
19	东城, 101133120581, 安定路, 88.09, 765, 86844
20	东城, 101134162614, 富贵园三区, 76.94, 540, 70185
21	东城, 101133897990, 化工大院, 36.55, 439, 120110
22	东城, 101132728861, 巷上嘉园, 103.37, 1069, 103415
23	东城, 101133354009, 和平里六区, 61.63, 646, 104820

图 4-2 爬取结果

完整内容见我一并提交到附件。

如果爬取过程中遇到失败情况，页面会保存到 `data/failed_pages` 下，便于分析，如图 3-5 所示。

5 作业总结与心得体会

在本次作业中，我系统性地实践了网络爬虫开发的完整流程。从前期的网页结构分析、请求头和 Cookie 的抓取，到中期的爬虫架构设计、代码模块化实现，再到后期的异常处理和数据存储，每个环节都让我对爬虫技术有了更深刻的认识。

在网页分析方面，我通过使用浏览器开发者工具（DevTools）分析网页结构、定位数据元素、观察网络请求，并分析网页分页逻辑，为后续的爬虫设计奠定了基础。

在技术实现方面，我基于 Python 丰富的库生态：使用 BeautifulSoup 进行 HTML 解析和 CSS 选择器定位，使用 Selenium 进行浏览器自动化操作并绕过基本的反爬检测；使用 requests 库发送 HTTP 请求并处理 Cookie 认证。此外，模块化的代码组织方式也让我体会到了良好架构设计对于代码可维护性的重要性。

本次作业中遇到的最大挑战是链家网站的反爬机制。网站会在检测到异常访问时触发人机验证，因此需要通过设置合理的请求间隔降低触发频率，并配置完整的请求头信息模拟真实浏览器访问；

虽然本次作业基本完成了既定目标，但仍有一些可以改进的地方。首先，可以考虑引入异步请求机制（如 aiohttp），在保持相同请求间隔的前提下提高爬取效率。其次，可以将爬虫部署为定时任务，实现数据的自动更新和增量爬取。此外，还可以加入数据清洗和去重逻辑，提高数据质量；以及增加更完善的日志系统，便于问题追踪和性能分析。

总的来说，本次作业不仅让我掌握了网络爬虫的核心技术，更重要的是培养了面对复杂问题时的分析能力以及运用 Python 语言解决问题的思维方式。这些经验将对我今后的学习和工作产生积极的影响。

6 附录A：提交材料结构说明

本次作业提交材料的目录结构如下：

1	.	
2	├─ data/	# 数据目录
3	│ └─ failed_pages/	# 失败页面保存目录
4	│ └─ 20251121_201426_chaoyang_pg3.html	
5	│ └─ ...	# 其他失败页面记录
6	└─ lianjia_ershoufang.csv	# 爬取结果数据文件
7		
8	├─ docs/	# 文档目录
9	│ └─ report.md	# 实验报告
10		
11	├─ src/	# 源代码目录
12	│ └─ main.py	# 主程序入口
13	│ └─ crawler.py	# 爬虫核心逻辑（页面获取、解析、存储）
14	│ └─ config.py	# 配置文件（URL、请求头、参数等）
15	│ └─ cookie_manager.py	# Cookie管理模块（加载、保存）
16	│ └─ auto_login.py	# 基于Selenium的半自动登录模块
17	│ └─ proxy_pool.py	# 代理池管理模块
18	│ └─ cookies.json	# Cookie持久化存储文件
19	│ └─ proxy_pool.json	# 代理池配置文件
20		
21	└─ pyproject.toml	# Python项目配置文件（uv包管理）
22	└─ uv.lock	# 依赖锁定文件
23	└─ README.md	# 项目说明文档

7 附录B：爬取文件的完整结果

1	城区,房源编号,小区名称,平米数(单位: 平米),总价(单位: 万),单价(单位: 元/平)
2	东城,101132675987,国瑞城东区,69.01,578,83756
3	东城,101133828145,新中西里,72.38,670,92568
4	东城,101129411869,广渠家园,79.55,650,81710
5	东城,101132480005,地坛北里,53.53,536,100131
6	东城,101131524357,广渠家园,60.79,465,76493
7	东城,101132603835,察慈小区,44.5,300,67416
8	东城,101134144805,地坛北门,52.61,499,94849
9	东城,101133592145,民旺园,55.27,535,96798
10	东城,101132150391,和平里五区,55.12,579,105044
11	东城,101134059409,化工大院,36.55,445,121752
12	东城,101132979778,西革新里112号院,88.69,699,78814
13	东城,101128711497,望坛新苑,65.62,496,75587
14	东城,101134142498,白桥大街,57.86,348,60146
15	东城,101134214716,左安门内大街,58.73,340,57893
16	东城,101132225367,冠城名敦道A区,85.03,520,61155
17	东城,101134172488,景泰西里西区,77.46,450,58095
18	东城,101133576612,鼎盛园,81.91,480,58601
19	东城,101133120581,安定路,88.09,765,86844
20	东城,101134162614,富贵园三区,76.94,540,70185
21	东城,101133897990,化工大院,36.55,439,120110
22	东城,101132728861,巷上嘉园,103.37,1069,103415
23	东城,101133354009,和平里六区,61.63,646,104820
24	东城,101133737265,百荣嘉园,56.73,348,61344
25	东城,101130824384,南竹杆胡同,68.84,699,101540
26	东城,101132848332,青年湖东里,38.27,396,103476
27	东城,101134155717,绿景苑,134.03,960,71626
28	东城,101133655762,东方财富,45.29,325,71760
29	东城,101131762628,春秀路,59.57,499,83767
30	东城,101134192040,竹杆胡同,72.23,549,76008
31	东城,101131768290,交林,41.62,418,100433
32	东城,101133293393,兴化西里,90.65,599,66079
33	东城,101134142498,白桥大街,57.86,348,60146
34	东城,101130592031,百荣嘉园,43.06,248,57595
35	东城,101130543801,西革新里110号院,60.51,249,41151
36	东城,101130696903,中海紫御公馆,61.03,485,79470
37	东城,101132041450,琉璃井路,66.29,420,63358
38	东城,101130824384,南竹杆胡同,68.84,699,101540
39	东城,101129374139,新奥洋房,154.24,1080,70021
40	东城,101133641596,海运仓小区,82.72,760,91877
41	东城,101131297528,宝华家园,89.51,485,54184
42	东城,101134192040,竹杆胡同,72.23,549,76008
43	东城,101132175200,安化北里,50.89,390,76636
44	东城,101130741229,广渠家园,71.27,438,61457
45	东城,101133949858,和平里七区,46.56,428,91925
46	东城,101134112137,十字坡西里,58.95,558,94657
47	东城,101134155717,绿景苑,134.03,960,71626
48	东城,101131768290,交林,41.62,418,100433
49	东城,101133984002,安外大街3号院,71.67,530,73951
50	东城,101128711497,望坛新苑,65.62,496,75587
51	东城,101132225367,冠城名敦道A区,85.03,520,61155

52	东城,101132979778,西革新里112号院,88.69,699,78814
53	东城,101134214716,左安门内大街,58.73,340,57893
54	东城,101133659219,中海紫御公馆,90.09,1020,113221
55	东城,101134040277,中海紫御公馆,90.08,828,91919
56	东城,101133980838,龙潭北里,62.92,480,76288
57	东城,101133385014,左安漪园,97.33,760,78085
58	东城,101132506538,小黄庄一区,69.27,630,90949
59	东城,101133267515,百荣嘉园,83.63,455,54407
60	东城,101134151910,赵堂子胡同,81.7,699,85557
61	东城,101133669925,远洋德邑,127.42,780,61215
62	东城,101134212584,西营房,85.07,658,77349
63	东城,101130596207,花园胡同,57.1,540,94571
64	东城,101133293393,兴化西里,90.65,599,66079
65	东城,101132591597,花梗胡同1号院,57.4,640,111499
66	东城,101129652985,朝阳门南小街,57.64,451,78245
67	东城,101133139544,化工大院,46.11,538,116678
68	东城,101134026062,东直门北大街,55.92,395,70637
69	东城,101134230903,雍和家园一期,267.32,1980,74069
70	东城,101131899000,南弓匠营胡同,87,920,105748
71	东城,101133632920,南岗子,51.13,370,72365
72	东城,101134186564,禾风仕家,78.55,548,69765
73	东城,101134150531,广渠家园,71.15,388,54533
74	东城,101132576493,广渠家园,51.12,313,61229
75	东城,101133981601,定安里,50.16,249,49642
76	东城,101129107928,忠实里,61.59,479,77773
77	东城,101133324038,光明西里,49.74,362,72779
78	东城,101133705583,冠城名敦道A区,68.19,360,52794
79	东城,101134179996,水上华城,62.65,570,90982
80	东城,101133945328,和平里五区,77.58,899,115881
81	东城,101132845362,胜古中路1号院,149.05,899,60316
82	东城,101133345702,西水井胡同,74.38,558,75021
83	东城,101133267515,百荣嘉园,83.63,455,54407
84	东城,101134151910,赵堂子胡同,81.7,699,85557
85	东城,101132807174,安外花园,73.4,598,81472
86	东城,101132710562,沙子口路70号,36.5,230,63014
87	东城,101131562009,城市亮点,124.25,789,63502
88	东城,101131098001,安德路55号院,56.69,430,75852
89	东城,101134077142,永定门东街西里,50.77,299,58894
90	东城,101134022995,豆瓣胡同,71.32,639,89597
91	东城,101133081907,上龙西里,80.28,798,99403
92	西城,101132845377,蝶翠华庭,106.33,1020,95928
93	西城,101132994214,红山世家,86.83,860,99045
94	西城,101130488856,什坊街,50.6,660,130435
95	西城,101133705231,铁一区,42.7,539,126230
96	西城,101130673571,红居南街,54.94,399,72625
97	西城,101134067312,五路通北街甲1号院,51.3,630,122808
98	西城,101130507363,爱民里小区,34.7,399,114986
99	西城,101134011320,远见名苑,83.1,669,80506
100	西城,101131272855,裕中西里,51.5,699,135729
101	西城,101132219914,月坛西街西里,51.6,665,128876
102	西城,101129535175,白菜湾四巷17号院,53.25,488,91644
103	西城,101133101091,三里河东路,68.44,610,89130
104	西城,101132954039,三里河南七巷1号院,92.03,1550,168424
105	西城,101128842285,团结大院,44.3,420,94809
106	西城,101134267330,远见名苑,81.37,665,81726

107	西城,101132668105,双旗杆东里,67.5,668,98963
108	西城,101131441201,裕中东里,42.3,625,147755
109	西城,101130190568,六铺炕一区6号院,58.8,639,108674
110	西城,101132684793,荣丰2008,28.44,273,95992
111	西城,101132287835,畅柳园,65.4,530,81040
112	西城,101132829418,新明胡同,60.7,698,114992
113	西城,101133801173,政泰家园,101.3,559,55183
114	西城,101131121196,南沙沟小区,59.95,850,141785
115	西城,101133272690,六铺炕一区,56.8,798,140493
116	西城,101131792433,大井胡同,38.7,530,136951
117	西城,101131664789,车公庄北里,78.9,745,94424
118	西城,101133938433,畅柳园,79.73,696,87295
119	西城,101134256614,太平里,53.37,340,63707
120	西城,101131123455,荣丰2008,33.98,265,77988
121	西城,101133279237,太平里,53.32,338,63391
122	西城,101129967468,安德馨居,66.64,858,128752
123	西城,101134174506,月坛西街西里,36.4,556,152748
124	西城,101133814676,远见国际公寓,48.79,298,61079
125	西城,101132615374,教场口9号院,73.29,750,102334
126	西城,101133007837,新明胡同,59.6,655,109900
127	西城,101131282444,月坛北街,40.8,580,142157
128	西城,101131070542,复兴门北大街,63.8,700,109718
129	西城,101133468513,人定湖西里,52.5,618,117715
130	西城,101133930808,百万庄生肖区,40.7,427,104915
131	西城,101131782158,马连道中街,53.49,338,63190
132	西城,101131121196,南沙沟小区,59.95,850,141785
133	西城,101133635260,荣丰2008,31.9,308,96552
134	西城,101130340680,月坛南街北里,70.5,780,110639
135	西城,101133999826,平原里小区,65.9,510,77390
136	西城,101131708451,远见名苑,44.6,309,69283
137	西城,101132893116,畅柳园,82.77,680,82156
138	西城,101133945853,新明胡同,48.2,558,115768
139	西城,101132610557,新兴中巷,51.7,398,76983
140	西城,101133210616,荣丰2008,25.14,235,93477
141	西城,101132232651,天宁寺西里,44.56,275,61715
142	西城,101126407809,小马厂西里,57.88,438,75674
143	西城,101131739435,真武庙六里,43.3,580,133950
144	西城,101131640284,鸭子桥路,39.31,265,67413
145	西城,101124795162,红山世家,90.95,928,102035
146	西城,101130341750,马连道西里,50.35,339,67329
147	西城,101130708805,平原里小区,62.74,510,81288
148	西城,101132029811,人定湖西里,46.4,590,127156
149	西城,101131943812,前半壁街,39.3,392,99746
150	西城,101133586433,裕中西里,72.4,900,124310
151	西城,101133901746,畅柳园,65.44,525,80227
152	西城,101133945853,新明胡同,48.2,558,115768
153	西城,101133210616,荣丰2008,25.14,235,93477
154	西城,101132232651,天宁寺西里,44.56,275,61715
155	西城,101131739435,真武庙六里,43.3,580,133950
156	西城,101126407809,小马厂西里,57.88,438,75674
157	西城,101130341750,马连道西里,50.35,339,67329
158	西城,101131640284,鸭子桥路,39.31,265,67413
159	西城,101124795162,红山世家,90.95,928,102035
160	西城,101130340680,月坛南街北里,70.5,780,110639
161	西城,101133635260,荣丰2008,31.9,308,96552

162	西城,101132610557,新兴中巷,51.7,398,76983
163	西城,101130708805,平原里小区,62.74,510,81288
164	西城,101128772464,新风南里,66.3,755,113877
165	西城,101133154652,爱民里小区,40.6,470,115764
166	西城,101132865787,马连道西里,81.62,635,77800
167	西城,101131874769,双旗杆东里,49.2,688,139838
168	西城,101132313156,远见名苑,48.96,295,60254
169	西城,101132029811,人定湖西里,46.4,590,127156
170	西城,101131943812,前半壁街,39.3,392,99746
171	西城,101133586433,裕中西里,72.4,900,124310
172	西城,101133901746,畅柳园,65.44,525,80227
173	西城,101131991207,阳光丽景,144.99,2220,153115
174	西城,101133992964,百万庄13号院,113.7,1280,112577
175	西城,101134069662,北礼士路北区,41.2,340,82525
176	西城,101129836205,塔院胡同12号院,57.3,500,87261
177	西城,101132092098,城市之光,103.15,500,48474
178	西城,101132800963,新兴中巷,50.4,429,85120
179	西城,101133180393,感化胡同3号院,56.6,499,88163
180	西城,101132946247,阳光丽景,111.73,1646,147320
181	西城,101130906636,新文化街,62.5,618,98880
182	海淀,101131783737,玉海园五里,77.14,450,58336
183	海淀,101132773711,万泉庄小区,66.5,599,90076
184	海淀,101133671411,铭科苑,66,350,53031
185	海淀,101131678115,橡树湾五期,237.7,2500,105175
186	海淀,101124346756,安宁里南区,95.1,417,43849
187	海淀,101133140279,定慧西里,82.3,435,52856
188	海淀,101133636367,二里庄北里,48.1,275,57173
189	海淀,101133058007,静淑苑,67.3,565,83953
190	海淀,101133493325,富海中心,128.46,998,77690
191	海淀,101129008873,怡美家园,99.41,599,60256
192	海淀,101128387259,橡树湾一期,81.23,710,87407
193	海淀,101133457343,北影小区,58.6,330,56314
194	海淀,101133679394,双榆树南里二区,75.26,598,79458
195	海淀,101132678232,永定路17号院,12.8,149,116407
196	海淀,101132527926,上地东里,78.35,780,99554
197	海淀,101133910621,国海广场,79.96,370,46274
198	海淀,101133273564,621小区,49.5,276,55758
199	海淀,101128848121,上地东里,70.44,679,96395
200	海淀,101129538898,大有庄100号院,92.98,948,101958
201	海淀,101133841627,永泰园,49.4,215,43523
202	海淀,101131601508,冠城大通百旺府,93.03,609,65463
203	海淀,101132482922,双紫小区,89.33,680,76123
204	海淀,101133251091,铁道科学研究院,79.61,899,112926
205	海淀,101133075593,玉海园五里,40.87,280,68510
206	海淀,101130763854,金隅美和园,82.23,499,60684
207	海淀,101131524190,海淀南路,61.8,510,82525
208	海淀,101134015314,文慧园,56.45,399,70683
209	海淀,101133725599,铁道部宿舍,76.84,498,64810
210	海淀,101133423916,晨月园,61.11,440,72002
211	海淀,101134111516,小南庄社区,41.74,260,62291
212	海淀,101133181963,橡树湾三期,90.25,686,76012
213	海淀,101134066867,门头馨村北二区,93.61,430,45936
214	海淀,101130609940,海淀南路34号,52.59,350,66553
215	海淀,101131949055,北新家园,59.2,208,35136
216	海淀,101133276166,定慧东里,66.3,365,55053

217	海淀,101133529579,万寿路电信宿舍,69.1,558,80753
218	海淀,101133566137,复兴路79号院,62.7,449,71611
219	海淀,101134074788,毛纺厂南小区,55.3,263,47559
220	海淀,101131473226,明光村小区,79.5,535,67296
221	海淀,101133933330,金沟河5号院,48.5,229,47217
222	海淀,101127347164,城华园,93.86,530,56468
223	海淀,101133916746,建材城东一里,60.01,270,44993
224	海淀,101133966598,恩济庄82号院,78.9,536,67935
225	海淀,101133933057,北蜂窝中路2号院,44.1,320,72563
226	海淀,101123798037,华天大厦,54.58,215,39392
227	海淀,101133110574,文晟家园南里,61.37,298,48558
228	海淀,101133918784,冶金大院,56.3,330,58615
229	海淀,101134002828,北洼西里,54.8,350,63869
230	海淀,101132346212,红联南村45号院,54.3,355,65378
231	海淀,101134186423,增光路甲51号院,42.22,248,58740
232	海淀,101133089285,新起点嘉园,112.72,1050,93152
233	海淀,101133086890,万寿路西街9号,96.9,899,92777
234	海淀,101134018708,文慧园一号公寓,74.45,575,77234
235	海淀,101133831451,明光北里,78.48,588,74924
236	海淀,101133743417,上地东里,69.19,650,93945
237	海淀,101129610282,凯盛家园,59.98,230,38347
238	海淀,101131589459,红联南村46号院,52.4,349,66604
239	海淀,101133443679,天合家园,84.96,575,67679
240	海淀,101134144492,橡树湾二期,80.59,669,83013
241	海淀,101133895095,志新村,61.6,399,64773
242	海淀,101131819191,西三环北路105号院,80.4,766,95274
243	海淀,101134270674,太阳园,101.06,995,98457
244	海淀,101128082072,中湾国际,63.81,355,55634
245	海淀,101132928357,观林园,98.21,658,67000
246	海淀,101133499123,橡林郡,88.63,760,85750
247	海淀,101133978225,水利局小区,50,279,55800
248	海淀,101129042026,西钓鱼台嘉园,123.83,1298,104822
249	海淀,101131670510,观景园,90.95,595,65421
250	海淀,101132654888,京泉馨苑东里,92.73,395,42597
251	海淀,101134137965,永泰园新地标,188.68,898,47594
252	海淀,101133474006,双榆树东里,54.1,370,68392
253	海淀,101133384370,沁春家园,86.46,358,41407
254	海淀,101134074436,明光村小区,50.6,328,64823
255	海淀,101134201498,中鑫嘉园,101.34,730,72035
256	海淀,101133707590,宝盛里,71.5,359,50210
257	海淀,101128515282,上地东里,78.1,798,102177
258	海淀,101127578013,枫润家园,93.74,840,89610
259	海淀,101132958637,永定路70号院,90.1,640,71033
260	海淀,101131149078,海淀南路,40.3,349,86601
261	海淀,101133040514,金沟河5号院,61.8,346,55988
262	海淀,101131633785,恩济里小区,52.7,298,56547
263	海淀,101129875188,蓟门北里,54.5,399,73212
264	海淀,101122685652,永泰园,50.06,219,43748
265	海淀,101133494451,红联北村,55.2,345,62500
266	海淀,101126283538,海淀南路,68.68,480,69890
267	海淀,101132221525,宝盛里,86.67,439,50652
268	海淀,101132578862,建材城东二里,73.5,265,36055
269	海淀,101133316662,交大东路4号院,55.34,338,61077
270	海淀,101133645447,立方庭,34.13,260,76180
271	海淀,101132795018,橡树湾二期,88.49,818,92440

272	朝阳,101133676400,双龙南里,42.02,147,34984
273	朝阳,101133333319,柏林爱乐二期,82.42,289,35065
274	朝阳,101133622078,金蝉北里,83.3,279,33494
275	朝阳,101130148267,甘露园南里,59.51,240,40330
276	朝阳,101134106245,晨光家园A区,57.81,320,55354
277	朝阳,101134117258,酷特区,61.18,219,35797
278	朝阳,101133106404,妇联小区,64.63,198,30636
279	朝阳,101134038693,南平里,67.1,175,26081
280	朝阳,101133388873,华纺易城,139.41,899,64487
281	朝阳,101132511248,融科橄榄城一期,93.57,725,77483
282	朝阳,101134022643,慧谷阳光,54.38,339,62340
283	朝阳,101132620800,金隅泰和园,79.68,316,39659
284	朝阳,101133120897,富力又一城C区,89.46,392,43819
285	朝阳,101133946162,望花路西里,83.92,398,47427
286	朝阳,101133790972,优点社区,93.6,322,34402
287	朝阳,101134079366,方恒国际中心,82.34,200,24290
288	朝阳,101132871278,世纪村西区,52.3,235,44934
289	朝阳,101130979571,炫特嘉园二期,59.33,272,45846
290	朝阳,101132538297,甘露园南里,59.51,209,35121
291	朝阳,101133885473,青年汇佳园,72.62,479,65960
292	朝阳,101133206749,华严北里小区,45.29,228,50343
293	朝阳,101133154925,燕保祈东家园二期,61.37,172,28027
294	朝阳,101133731507,望京新城,91.62,350,38202
295	朝阳,101133066428,后现代城A区,47.72,198,41493
296	朝阳,101133266988,东方东路,41.11,199,48407
297	朝阳,101133549017,八里庄北里小区,60.38,198,32793
298	朝阳,101133954376,团结湖北头条,61.1,349,57120
299	朝阳,101133264878,砖角楼南里,56.36,340,60327
300	朝阳,101132767877,京师园,90.98,770,84634
301	朝阳,101130136488,澳洲康都,133.46,899,67362
302	朝阳,101132767877,京师园,90.98,770,84634
303	朝阳,101134161210,华严北里小区,70.42,470,66743
304	朝阳,101133967228,花家地南里,71.82,330,45949
305	朝阳,101132626096,世安家园,82.11,398,48472
306	朝阳,101132492136,小黄庄,48.21,280,58080
307	朝阳,101133394521,和平西街,53.81,321,59655
308	朝阳,101133175292,沿海赛洛城二期,93.66,368,39292
309	朝阳,101133895297,金隅丽港城,54.8,255,46533
310	朝阳,101134146346,团结湖北头条,62.69,285,45462
311	朝阳,101133542284,世茂奥临花园,172.07,1239,72006
312	朝阳,101132514277,八里庄南里,59.5,249,41849
313	朝阳,101133505325,奥林匹克花园三期,126.16,999,79186
314	朝阳,101132736748,国美第一城3号院,57.86,235,40616
315	朝阳,101133902558,远洋天地三期,91.27,460,50400
316	朝阳,101122712988,乐成豪丽,70.4,399,56677
317	朝阳,101133880932,花家地西里三区,39.14,215,54932
318	朝阳,101133111542,曙光西里,60.81,278,45717
319	朝阳,101133762233,方舟苑一期,86.31,380,44028
320	朝阳,101132347506,国美第一城3号院,92.13,425,46131
321	朝阳,101132064542,金隅汇星苑,79.01,399,50500
322	朝阳,101133630561,北京新天地二期,67.08,269,40102
323	朝阳,101133478198,甘露园南里,110.79,420,37910
324	朝阳,101133413246,润泽悦溪,89.11,645,72383
325	朝阳,101127924474,格林莱雅,149.33,700,46877
326	朝阳,101131531434,慧忠里B区,44.64,368,82438

327	朝阳,101130020590,劲松四区,54.72,219,40022
328	朝阳,101128682856,现代城,73.18,399,54524
329	朝阳,101130084193,金隅汇星苑,82.54,350,42404
330	朝阳,101132753621,兴隆家园,105.47,535,50726
331	朝阳,101133855243,望京新城,105.98,460,43405
332	朝阳,101133762233,方舟苑一期,86.31,380,44028
333	朝阳,101132347506,国美第一城3号院,92.13,425,46131
334	朝阳,101132064542,金隅汇星苑,79.01,399,50500
335	朝阳,101133111542,曙光西里,60.81,278,45717
336	朝阳,101129839311,兴隆家园,91.52,328,35840
337	朝阳,101133744238,炫特嘉园三期,91.08,498,54678
338	朝阳,101127109033,呼家楼北街,64.5,396,61396
339	朝阳,101133875294,金隅景和园六区,87.11,329,37769
340	朝阳,101133978873,后现代城B区,50.17,192,38270
341	朝阳,101133413246,润泽悦溪,89.11,645,72383
342	朝阳,101131531434,慧忠里B区,44.64,368,82438
343	朝阳,101134021017,凯景铭座,122.73,710,57851
344	朝阳,101132295403,方舟苑二期,98.72,460,46597
345	朝阳,101130020590,劲松四区,54.72,219,40022
346	朝阳,101133386782,保利锦上二期,94.12,610,64811
347	朝阳,101127924474,格林莱雅,149.33,700,46877
348	朝阳,101134178005,华贸城,53.04,399,75227
349	朝阳,101132456468,白家庄东里,62.79,323,51442
350	朝阳,101133702931,惠新里小区,69.53,525,75507
351	朝阳,101134015088,燕保祈东家园,94.93,285,30023
352	朝阳,101133658848,安贞西里,37.87,269,71033
353	朝阳,101133535562,周家庄中路19号院,56.53,180,31842
354	朝阳,101129244613,金隅汇星苑,87.95,396,45026
355	朝阳,101132818196,富力又一城C区,89.46,375,41919
356	朝阳,101134300002,高杨树北里,58.97,198,33577
357	朝阳,101134042434,望京西园三区,102.51,535,52191
358	朝阳,101131348908,华瀚福园C区,54.7,135,24681
359	朝阳,101134113494,十里堡,55.83,210,37615
360	朝阳,101133855243,望京新城,105.98,460,43405
361	朝阳,101130084193,金隅汇星苑,82.54,350,42404

8 附录C：爬虫主程序代码实现

由于篇幅限制，此处仅展示爬虫主程序 `crawler.py` 的完整代码：

```
1  # -*- coding: utf-8 -*-
2  """
3  爬虫主程序
4  """
5
6  import requests
7  from bs4 import BeautifulSoup
8  import csv
9  import re
10 import time
11 import os
12 from datetime import datetime
13 from config import (
14     BASE_URL, DISTRICTS, PAGES_PER_DISTRICT,
15     HEADERS, REQUEST_DELAY, OUTPUT_FILE, CSV_COLUMNS,
16     MAX_CONSECUTIVE_FAILURES, FAILED_PAGES_DIR,
17     PAGE_RETRY_COUNT, RETRY_DELAY
18 )
19 from cookie_manager import load_cookies_from_file
20 from proxy_pool import ProxyPool, fetch_with_proxy
21
22 def build_url(district_code: str, page: int) -> str:
23     """
24     构建目标URL
25
26     Args:
27         district_code: 城区代码, 如 'haidian'
28         page: 页码, 从1开始
29
30     Returns:
31         完整的URL字符串
32     """
33     if page == 1:
34         return f"{BASE_URL}{district_code}/"
35     return f"{BASE_URL}{district_code}/pg{page}/"
36
37
38 def parse_area(house_info: str) -> str:
39     """
40     从房屋信息中解析平米数
41
42     Args:
43         house_info: 房屋信息字符串, 如 "2室1厅 | 89.3平米 | 南 | 精装 | 中楼层"
44
45     Returns:
46         平米数字符串, 如 "89.3"
47     """
48     match = re.search(r'([\d.]+)平米', house_info)
```

```

49         if match:
50             return match.group(1)
51         return ""
52
53
54 def parse_house_id(href: str) -> str:
55     """
56     从链接中解析房源编号
57
58     Args:
59         href: 房源链接, 如 "/ershoufang/101123456789.html"
60
61     Returns:
62         房源编号, 如 "101123456789"
63     """
64     match = re.search(r'/ershoufang/(\d+)\.html', href)
65     if match:
66         return match.group(1)
67     return ""
68
69
70 def parse_unit_price(price_text: str) -> str:
71     """
72     解析单价文本
73
74     Args:
75         price_text: 单价文本, 如 "单价65123元/平米" 或 "53,031元/平"
76
77     Returns:
78         单价数字, 如 "65123" 或 "53031" (移除逗号)
79     """
80     # 移除所有逗号, 然后提取数字
81     clean_text = price_text.replace(',', '')
82     match = re.search(r'(\d+)', clean_text)
83     if match:
84         return match.group(1)
85     return ""
86
87
88 def save_failed_page(html: str, url: str, reason: str, status_code: int =
None):
89     """
90     保存失败的页面到本地, 便于分析
91
92     Args:
93         html: 页面HTML内容
94         url: 请求的URL
95         reason: 失败原因
96         status_code: HTTP状态码 (如有)
97     """
98     os.makedirs(FAILED_PAGES_DIR, exist_ok=True)
99
100     timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
101     # 从URL中提取城区和页码

```



```

102     url_part = url.replace(BASE_URL, "").replace("/", "_").strip("_") or
    "index"
103     filename = f"{timestamp}_{url_part}.html"
104     filepath = os.path.join(FAILED_PAGES_DIR, filename)
105
106     # 添加调试信息
107     debug_info = f"""<!--
108 失败页面调试信息
109 =====
110 URL: {url}
111 时间: {datetime.now().strftime("%Y-%m-%d %H:%M:%S")}
112 状态码: {status_code or "N/A"}
113 失败原因: {reason}
114 =====
115 -->
116 """
117
118     with open(filepath, 'w', encoding='utf-8') as f:
119         f.write(debug_info + (html or "<!-- 页面内容为空 -->"))
120
121     print(f" ⚠️ 失败页面已保存: {filepath}")
122
123
124 def fetch_page_with_retry(url: str, cookies: dict = None, proxy_pool:
ProxyPool = None) -> tuple:
125     """
126     带重试机制的页面获取
127
128     Args:
129         url: 目标URL
130         cookies: 可选的cookies字典
131         proxy_pool: 可选的代理池实例
132
133     Returns:
134         (页面HTML内容, HTTP状态码, 错误信息, 重试次数)
135     """
136     last_html, last_status, last_error = "", None, None
137
138     for attempt in range(PAGE_RETRY_COUNT + 1):
139         html, status_code, error = fetch_page(url, cookies, proxy_pool)
140
141         # 请求成功且状态码正常
142         if status_code == 200 and html:
143             return html, status_code, None, attempt
144
145         # 记录最后一次的结果
146         last_html, last_status, last_error = html, status_code, error
147
148         # 如果还有重试机会
149         if attempt < PAGE_RETRY_COUNT:
150             print(f" 🔄 第 {attempt + 1} 次重试 (等待 {RETRY_DELAY} 秒) ...")
151             time.sleep(RETRY_DELAY)
152
153     # 所有重试都失败
154     return last_html, last_status, last_error, PAGE_RETRY_COUNT

```

```

155
156
157 def fetch_page(url: str, cookies: dict = None, proxy_pool: ProxyPool = None)
    -> tuple:
158     """
159     获取页面HTML内容（支持代理池）
160
161     Args:
162         url: 目标URL
163         cookies: 可选的cookies字典
164         proxy_pool: 可选的代理池实例
165
166     Returns:
167         (页面HTML内容, HTTP状态码, 错误信息)
168     """
169     try:
170         if proxy_pool:
171             # 使用代理池
172             response = fetch_with_proxy(
173                 url,
174                 proxy_pool,
175                 headers=HEADERS,
176                 cookies=cookies,
177                 timeout=10,
178                 max_retries=3
179             )
180             if response:
181                 return response.text, response.status_code, None
182             else:
183                 return "", None, "代理池请求失败"
184         else:
185             # 直连
186             response = requests.get(url, headers=HEADERS, cookies=cookies,
187                                     timeout=10)
188             return response.text, response.status_code, None
189     except requests.RequestException as e:
190         return "", None, str(e)
191
192 def parse_page(html: str, district_name: str) -> list:
193     """
194     解析页面，提取房源数据
195
196     Args:
197         html: 页面HTML内容
198         district_name: 城区名称
199
200     Returns:
201         房源数据列表，每个元素为字典
202     """
203     houses = []
204     soup = BeautifulSoup(html, 'html.parser')
205
206     # 查找房源列表容器
207     list_content = soup.select_one('ul.sellListContent')

```

```

208     if not list_content:
209         print("未找到房源列表容器")
210         return houses
211
212     # 遍历每个房源项
213     for item in list_content.select('li.clear'):
214         try:
215             house = {}
216             house['城区'] = district_name
217
218             # 房源编号: 从链接中提取
219             title_link = item.select_one('.title a')
220             if title_link and title_link.get('href'):
221                 house['房源编号'] = parse_house_id(title_link['href'])
222             else:
223                 house['房源编号'] = ""
224
225             # 小区名称
226             position_info = item.select_one('.positionInfo a')
227             house['小区名称'] = position_info.text.strip() if position_info
228         else: ""
229
230         # 平米数: 从houseInfo中解析
231         house_info = item.select_one('.houseInfo')
232         house['平米数 (单位: 平米)'] = parse_area(house_info.text) if
233         house_info else ""
234
235         # 总价
236         total_price = item.select_one('.totalPrice span')
237         house['总价 (单位: 万)'] = total_price.text.strip() if total_price
238         else ""
239
240         # 单价
241         unit_price = item.select_one('.unitPrice span')
242         house['单价 (单位: 元/平)'] = parse_unit_price(unit_price.text) if
243         unit_price else ""
244
245         # 只添加有效数据
246         if house['房源编号']:
247             houses.append(house)
248
249         except Exception as e:
250             print(f"解析房源项失败: {e}")
251             continue
252
253     return houses
254
255 def save_to_csv(data: list, filepath: str):
256     """
257     保存数据到CSV文件
258
259     Args:
260         data: 房源数据列表
261         filepath: 输出文件路径

```

```

259     """
260     # 确保目录存在
261     os.makedirs(os.path.dirname(filepath), exist_ok=True)
262
263     with open(filepath, 'w', newline='', encoding='utf-8-sig') as f:
264         writer = csv.DictWriter(f, fieldnames=CSV_COLUMNS)
265         writer.writeheader()
266         writer.writerows(data)
267
268     print(f"数据已保存到: {filepath}")
269
270
271 def crawl(cookies: dict = None, use_proxy: bool = False) -> tuple:
272     """
273     执行爬虫主逻辑
274
275     Args:
276         cookies: 可选的cookies字典, 用于登录态
277         use_proxy: 是否使用代理池
278
279     Returns:
280         (房源数据列表, 是否需要重新登录)
281     """
282     all_houses = []
283     consecutive_failures = 0
284     total_failures = 0
285     need_relogin = False
286
287     # 初始化代理池
288     proxy_pool = None
289     if use_proxy:
290         print("\n初始化代理池...")
291         proxy_pool = ProxyPool()
292         if proxy_pool.proxies:
293             print(f"代理池已就绪, 共 {len(proxy_pool.proxies)} 个代理")
294         else:
295             print("⚠️ 代理池为空, 将使用直连")
296             use_proxy = False
297
298     for district_name, district_code in DISTRICTS.items():
299         print(f"\n开始爬取: {district_name}")
300
301         for page in range(1, PAGES_PER_DISTRICT + 1):
302             url = build_url(district_code, page)
303
304             proxy_info = ""
305             if use_proxy and proxy_pool:
306                 proxy_info = " [使用代理]"
307
308             print(f" 正在爬取第 {page} 页{proxy_info}: {url}")
309
310             html, status_code, error, retries = fetch_page_with_retry(url,
311 cookies, proxy_pool if use_proxy else None)
312
313             # 显示重试信息

```

```

313         if retries > 0 and status_code == 200:
314             print(f"    ✓ 第 {retries + 1} 次尝试成功")
315
316         # 检查HTTP状态码
317         if status_code and status_code != 200:
318             reason = f"HTTP状态码错误: {status_code}"
319             print(f"    ✗ {reason}")
320             save_failed_page(html, url, reason, status_code)
321             consecutive_failures += 1
322             total_failures += 1
323
324         # 检查是否可能是Cookie失效（常见状态码：302重定向到登录页、403禁止）
325         if status_code in [302, 403, 401]:
326             print(f"状态码 {status_code} 可能表示Cookie已失效")
327             need_relogin = True
328
329         elif error:
330             reason = f"请求错误: {error}"
331             print(f"    ✗ {reason}")
332             save_failed_page(html, url, reason)
333             consecutive_failures += 1
334             total_failures += 1
335
336         elif not html:
337             reason = "获取到空页面"
338             print(f"    ✗ {reason}")
339             save_failed_page("", url, reason)
340             consecutive_failures += 1
341             total_failures += 1
342
343         else:
344             # 解析页面
345             houses = parse_page(html, district_name)
346
347             if not houses:
348                 # 获取到页面但没有解析出数据
349                 reason = "未解析到房源数据（可能是Cookie失效）"
350                 print(f"    ⚠ {reason}")
351                 save_failed_page(html, url, reason, status_code)
352                 consecutive_failures += 1
353                 total_failures += 1
354
355                 # 如果是首页就没数据，很可能是Cookie失效
356                 if page == 1:
357                     print(f"    首页无数据，Cookie可能已失效")
358                     need_relogin = True
359                 else:
360                     # 成功获取数据，重置连续失败计数
361                     all_houses.extend(houses)
362                     print(f"    ✓ 获取到 {len(houses)} 条房源数据")
363                     consecutive_failures = 0
364
365             # 检查是否需要停止爬取
366             if consecutive_failures >= MAX_CONSECUTIVE_FAILURES:
367                 print(f"\n连续失败 {consecutive_failures} 次，停止爬取")

```

```
368
369         if need_relogin:
370             print(f"\n检测到可能的Cookie失效, 建议重新登录获取Cookie")
371
372         # 保存已获取的数据
373         if all_houses:
374             save_to_csv(all_houses, OUTPUT_FILE)
375             print(f"\n已保存 {len(all_houses)} 条已获取的数据")
376
377         return all_houses, need_relogin
378
379         # 请求间隔, 避免触发反爬
380         time.sleep(REQUEST_DELAY)
381
382     print(f"\n爬取完成, 共获取 {len(all_houses)} 条房源数据")
383     if total_failures > 0:
384         print(f"共有 {total_failures} 次失败, 失败页面已保存到:
385 {FAILED_PAGES_DIR}")
386
387     # 显示代理统计
388     if use_proxy and proxy_pool:
389         proxy_pool.show_statistics()
390
391     # 保存到csv
392     if all_houses:
393         save_to_csv(all_houses, OUTPUT_FILE)
394
395     return all_houses, need_relogin
```